

MULTITHREADED LINUX APPLICATION USING POSIX THREADS AND MUTEXES

LINUX SYSTEM MONITORING APPLICATION

Team Members :

2500031613 - U.Moksha

2520030408 - Jamuna

2520030416 - Keerthi

Section : 7

Team no. : 5

Faculty Name :

K . Hema

INTRODUCTION

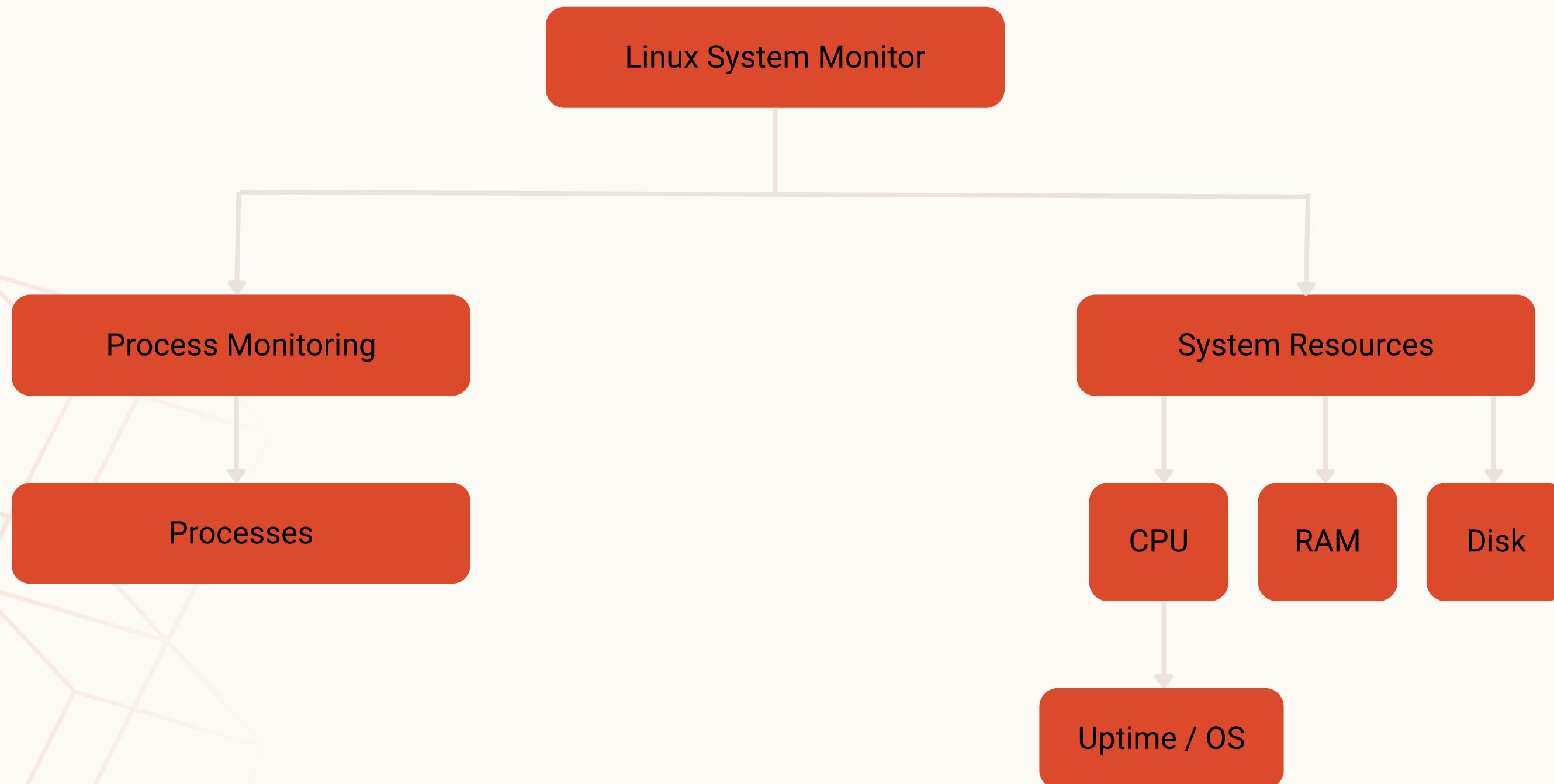
- A Linux system contains many processes and system resources.
- Monitoring these resources helps us understand the current state of the system.
- Our project provides a simple command-line application for monitoring Linux.
- The application uses POSIX threads for concurrent monitoring.
- Mutexes are used to synchronize access to shared data.

The project is intended to demonstrate concurrency, shared resources, critical sections, and synchronization.

OBJECTIVES

- Monitor running processes
- Monitor CPU usage
- Monitor RAM usage
- Monitor disk usage
- Display system uptime
- Display operating system information
- Display kernel version
- Demonstrate multithreading
- Demonstrate mutex synchronization
- Prevent race conditions
- The system-resource module specifically covers CPU, memory, disk, uptime, and OS/kernel information.

SYSTEM ARCHITECTURE



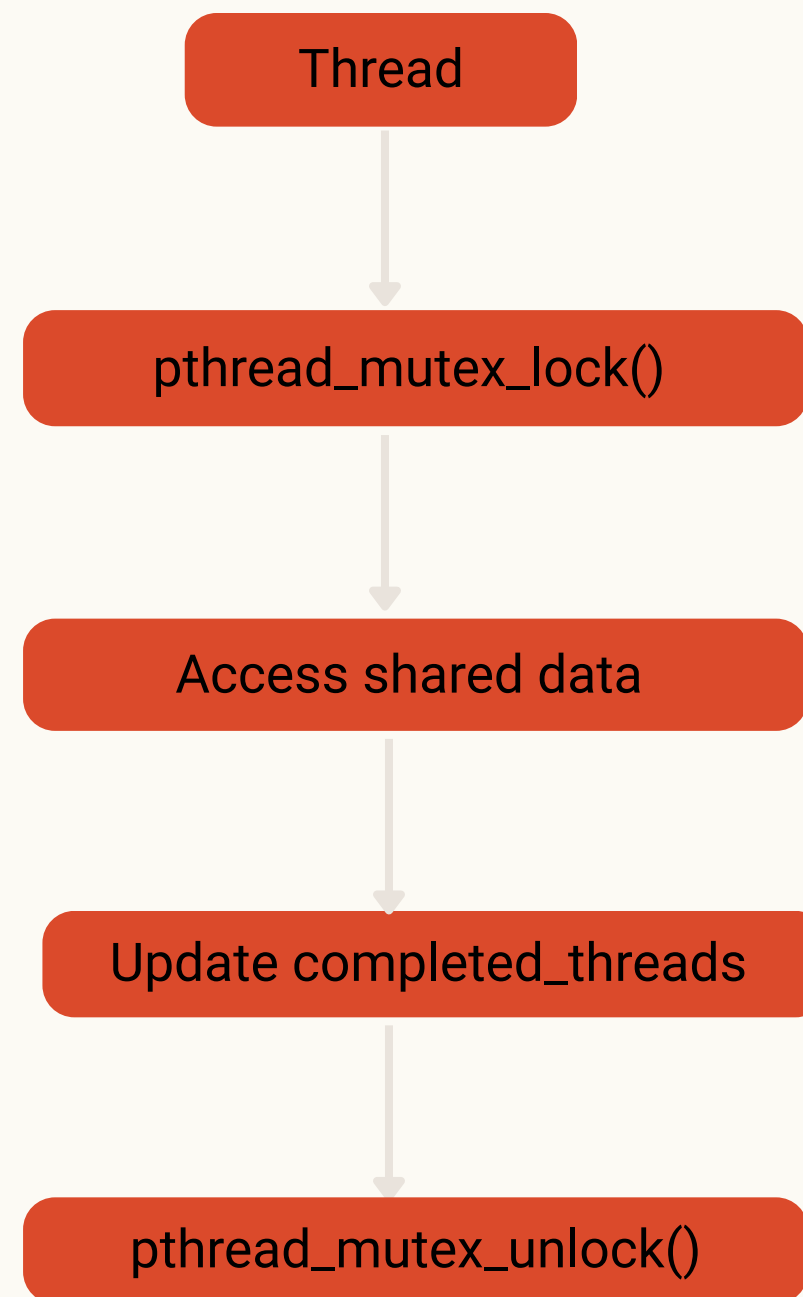
MULTITHREADING

Multithreading allows different tasks to be performed using separate threads within the same program.

Thread	Task
Thread 1	CPU Monitoring
Thread 2	RAM Monitoring
Thread 3	Disk Monitoring
Thread 4	Uptime Monitoring
Thread 5	OS/Kernal information

The project requires `pthread_create()` and `pthread_join()` for thread creation and synchronization.

MUTEX AND SYNCHRONIZATION



IMPLEMENTATION

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include "process_monitor.h"

void show_processes()
{
    printf("\n");
    printf("=====\n");
    printf("          RUNNING PROCESSES\n");
    printf("=====\n");

    system("ps -eo pid,stat,comm,%cpu,%mem --sort=-%cpu | head -n 11");
}
```

```
GNU nano 8.7.1
#ifndef PROCESS_MONITOR_H
#define PROCESS_MONITOR_H

void show_processes();

#endif
```

```
GNU nano 8.7.1
#ifndef SYSTEM_RESOURCES_H
#define SYSTEM_RESOURCES_H

void *cpu_usage(void *arg);
void *memory_usage(void *arg);
void *disk_usage(void *arg);
void *uptime(void *arg);
void *os_info(void *arg);

int get_completed_threads();
void reset_completed_threads();

#endif
```

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include "system_resources.h"

/* Mutex protects shared data and output */
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

/* Shared resource used by all threads */
int completed_threads = 0;
```

OUTPUT

```
=====
      LINUX SYSTEM MONITOR
=====
1. Process Monitoring
2. System Resources
3. Refresh
4. Exit
=====
Enter choice: 1

=====
      RUNNING PROCESSES
=====
  PID STAT  COMMAND          %CPU %MEM
   84 Ss    systemd-udev    0.0  0.1
    1 Ss    systemd         0.0  0.1
   46 S<s    systemd-journal 0.0  0.2
 2597 Ss     bash           0.0  0.0
 2593 S      Relay(2597)  0.0  0.0
   167 Ss    networkd-dispat 0.0  0.3
   440 Ss    systemd        0.0  0.1
   273 Ssl   unattended-upgr  0.0  0.4
   277 S      chronyd     0.0  0.1
   213 Ssl   rsyslogd        0.0  0.0
```

```
=====
      LINUX SYSTEM MONITOR
=====
1. Process Monitoring
2. System Resources
3. Refresh
4. Exit
=====
Enter choice: 2

CPU Usage       : 1.6
RAM Usage       : 548Mi / 7.6Gi
Disk Usage      : 1%
System Uptime   : up 1 hour, 55 minutes
Operating System : "Ubuntu 26.04 LTS"
Kernel Version  : 6.18.33.2-microsoft-standard-WSL2

Threads completed: 5/5
```

OUTPUT

```
=====
      LINUX SYSTEM MONITOR
=====
1. Process Monitoring
2. System Resources
3. Refresh
4. Exit
=====
Enter choice: 3
Refreshing...

CPU Usage      : 0
RAM Usage      : 549Mi / 7.6Gi
Disk Usage     : 1%
System Uptime  : up 1 hour, 56 minutes
Operating System : "Ubuntu 26.04 LTS"
Kernel Version  : 6.18.33.2-microsoft-standard-WSL2

Threads completed: 5/5
```

```
=====
      LINUX SYSTEM MONITOR
=====
1. Process Monitoring
2. System Resources
3. Refresh
4. Exit
=====
Enter choice: 4

Exiting program...
jamuna@Jem:~/os-monitor$ |
```

CONCLUSION

- The project successfully developed a Linux System Monitoring Application for monitoring important system resources and running processes.
- The application displays CPU usage, RAM usage, disk usage, system uptime, operating system, and kernel information.
- POSIX threads are used to monitor different system resources through separate threads.
- Mutex synchronization is used to protect shared data and help prevent race conditions.
- The project demonstrates important Operating System concepts, including multithreading, synchronization, shared resources, critical sections, and process monitoring.
- The application provides a simple menu-based interface that allows users to view system information and refresh the displayed results.
- Overall, the project shows how Linux system information and OS-level concepts can be combined in a practical multithreaded application.

THANK YOU